

SJSU 2026 Fall CMPE 180A Week 5 Term Project Intro

Prof. Sanja Damjanovic

sanja.damjanovic@sjsu.edu

Table of Contents

Reference tables	2
Projects overview	2
Open datasets	2
Suggested term project stages.....	4
Term project outputs:	4
General references (all projects)	4
Term Project Catalog (Fall 2026)	5
Group A — Graphs & Traversal	6
A1. Flight Route Planner	6
A2. University Course Planner	6
A3. Knowledge Graph Explorer.....	7
A4. Social Network Analysis.....	8
Group B — Hashing, Tries & Text Processing.....	9
B1. Autocomplete & Fuzzy Spell-Checker.....	9
B2. Document Search Engine	9
Group C — Heaps, Ranking & Recommendation	10
C1. Movie Recommender with k-Nearest-Neighbors.....	10
Group D — Linked Structures & Caching.....	11
D1. Build Your Own Cache (LRU / LFU)	11
Group E — Streaming & Windowed Analytics	12
E1. Real-Time Taxi-Stream Analytics	12
E2. Chat Analytics System	12

SJSU 2026 Fall CMPE 180A Week 5 Term Project Intro

Prof. Sanja Damjanovic

sanja.damjanovic@sjsu.edu

Reference tables

Projects overview

#	Project	Group / DSA area	New concept to self-learn
A1	Flight Route Planner	Graphs & traversal	Dijkstra + A* with own heap
A2	University Course Planner	Graphs & traversal	Kahn topo sort + cycle detection
A3	Knowledge Graph Explorer	Graphs & traversal	BFS shortest-path reconstruction + cycle detection
A4	Social Network Analysis	Graphs & traversal	PageRank + connected components
B1	Autocomplete & Spell-Checker	Hashing, tries & text	Trie + edit-distance DP
B2	Document Search Engine	Hashing, tries & text	Inverted index + trie + heap top- k
C1	Movie Recommender	Heaps & recommendation	Collaborative filtering + k-d tree
D1	LRU / LFU Cache	Linked structures & caching	$O(1)$ LRU and $O(1)$ LFU
E1	Taxi-Stream Analytics	Streaming & windowed	Two-heap median + Fenwick tree
E2	Chat Analytics System	Streaming & windowed	Streaming top- k + sliding window

Open datasets

Project	Dataset	Access
A1 Flight Route Planner	OpenFlights airports + routes	https://openflights.org/data.php (direct CSV)
A2 University	Instructor curriculum	Provided, or convert a public CS/AI catalog

SJSU 2026 Fall CMPE 180A Week 5 Term Project Intro

Prof. Sanja Damjanovic

sanja.damjanovic@sjsu.edu

Course Planner	CSV / public degree catalog	
A3 Knowledge Graph Explorer	ConceptNet English subset	https://github.com/commonsense/conceptnet5/wiki/Downloads
A4 Social Network Analysis	SNAP Facebook Social Circles (4,039 nodes)	https://snap.stanford.edu/data/egonets-Facebook.html
B1 Autocomplete & Spell-Checker	Norvig big.txt + SymSpell frequency list	https://norvig.com/big.txt · https://github.com/wolfgarbe/SymSpell
B2 Document Search Engine	Project Gutenberg .txt books / 20 Newsgroups	https://www.gutenberg.org · http://qwone.com/~jason/20Newsgroups/
C1 Movie Recommender	MovieLens ml-latest-small	https://grouplens.org/datasets/movielens/
D1 LRU / LFU Cache	Wikimedia pageview counts (one hour) + synthetic Zipfian	https://dumps.wikimedia.org/other/pageviews/
E1 Taxi-Stream Analytics	NYC TLC Trip Records (one	https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page

SJSU 2026 Fall CMPE 180A Week 5 Term Project Intro

Prof. Sanja Damjanovic

sanja.damjanovic@sjsu.edu

	month, Parquet)	
E2 Chat Analytics System	Cornell Movie- Dialogs Corpus	https://convokit.cornell.edu/documentation/movie.html

Suggested term project stages

Stage	Milestone
Stage 1 — Foundation	Loader + core structure + stats + tests; one-page proposal approved
Stage 2 — Core algorithm	Primary algorithm working + tests + library validation
Stage 3 — Stretch + features	Self-learn concept implemented + secondary features
Stage 4 — Evaluation	Runtime study + complexity write-up + visualization + report + demo

Term project outputs:

- Correct from-scratch structure + required algorithm(s)
- Stretch concept implemented and clearly explained
- Unit tests + correctness on known small inputs
- Complexity analysis + empirical runtime study
- Data loading/validation + one visualization
- Jupyter notebook + slides + demo/presentation

Libraries remain restricted to I/O, plotting, and validation.

General references (all projects)

- M. T. Goodrich, R. Tamassia & M. H. Goldwasser, *Data Structures and Algorithms in Python*, Wiley, 2013 (ISBN 978-1-118-29027-9) — **primary course textbook**.

SJSU 2026 Fall CMPE 180A Week 5 Term Project Intro

Prof. Sanja Damjanovic

sanja.damjanovic@sjsu.edu

Term Project Catalog (Fall 2026)

This catalog collects **ten** term-project options in one place, grouped by the data-structure area each one exercises. Every project is anchored in material from the course but deliberately requires **one structure or algorithm you must self-learn**; that “stretch concept” is where most of the learning (and grading weight) sits. Each team picks **one** project and builds it over the term in four defined stages.

How to read an entry

- *Course anchors* — the structures from lecture you’ll rely on.
- *Stretch concept (new)* — the one thing you must learn on your own and implement from scratch.
- *Problem* — what the finished program does.
- *Dataset* — a convenient, openly downloadable dataset (direct links in the master table at the end).
- *Implementation stages* — four milestones that carry you from an empty repo to a demo.
- *Outcomes* — what you submit and present

Rules that apply to every project

- The core structure/algorithm is implemented **from scratch** with Python dictionaries, sets, lists, deque, and (where noted) heapq. Libraries such as NetworkX, pandas, or scikit-learn are allowed only for loading, plotting, and **validating** your own results — never as a substitute for the required algorithm.
- Every project needs **unit tests** on small inputs with known answers (including empty, single-element, cyclic/duplicate, and disconnected cases), a **Big-O analysis** of your own code, and an **empirical runtime study** (at least one plot of time vs. input size).
- A Jupyter notebook implementation is sufficient. (Alternatively, a command-line interface is sufficient; a web UI is never required or rewarded.)

The four stages (same rhythm for all ten)

- **Stage 1 — Foundation:** data loader + core data structure + basic statistics + unit tests for the structure.
- **Stage 2 — Core algorithm:** the primary required algorithm(s), tested, and validated against a library reference.
- **Stage 3 — Stretch + features:** the new self-learn concept fully built, plus the project’s secondary features.

SJSU 2026 Fall CMPE 180A Week 5 Term Project Intro

Prof. Sanja Damjanovic

sanja.damjanovic@sjsu.edu

- **Stage 4 — Evaluation:** runtime experiments, complexity write-up, one visualization, final report and demo.
-

Group A — Graphs & Traversal

A1. Flight Route Planner

Course anchors: graphs, adjacency lists, queues, hashmaps **Stretch concept (new):** **Dijkstra with your own min-heap** and **A* search with an admissible haversine heuristic**.

Problem. Given the world airline network, find the cheapest / shortest / fewest-stop route between two airports, and show how informed search (A*) expands fewer nodes than uninformed search (BFS/Dijkstra) while returning the same answer.

Dataset. OpenFlights airports + routes (~3,300 airports, ~67,000 routes, with coordinates) — direct CSV download.

Implementation stages. 1. *Foundation* — load `airports.dat/routes.dat` into an adjacency-list graph; compute great-circle edge weights (haversine); report graph statistics; test on a hand-built mini-graph. 2. *Core algorithm* — BFS for minimum hops; validate reachability against NetworkX. 3. *Stretch + features* — Dijkstra using a heap you implement, then A* with the haversine heuristic; argue admissibility. 4. *Evaluation* — compare nodes expanded and runtime for BFS vs. Dijkstra vs. A* on long-haul queries; render one route on a map; complexity write-up and report.

Outcomes. A RoutePlanner supporting hops/distance queries, a BFS-vs-Dijkstra-vs-A* comparison table, a rendered route map, and tests.

Main references. Goodrich, Tamassia & Goldwasser, *DSA in Python* — **Ch. 14 Graph Algorithms** (shortest paths), **Ch. 9 Priority Queues**, **Ch. 6 Queues**. · Hart, Nilsson & Raphael, “A Formal Basis for the Heuristic Determination of Minimum Cost Paths,” *IEEE Trans. SSC*, 1968. · CLRS, 4th ed. — Dijkstra and BFS.

A2. University Course Planner

Course anchors: directed graphs, adjacency lists, hash maps, queues, DFS. **Stretch concept (new):** **topological sorting (Kahn’s algorithm)** and **directed-cycle detection**.

SJSU 2026 Fall CMPE 180A Week 5 Term Project Intro

Prof. Sanja Damjanovic

sanja.damjanovic@sjsu.edu

Problem. Model course prerequisites as a directed graph, verify it is acyclic, produce a valid course order, and generate a semester-by-semester plan under a credit limit. Report and explain any cycle.

Dataset. Instructor-provided curriculum CSV (~15–40 courses + prerequisites, plus one cyclic file for testing), or a small public CS/AI degree catalog converted to CSV.

Implementation stages. 1. *Foundation* — load/validate the CSVs (unknown references, duplicate edges, self-prerequisites); build adjacency list + in-degree map; test on a known small curriculum. 2. *Core algorithm* — Kahn’s topological sort; flag a cycle when the ordering is short. 3. *Stretch + features* — DFS cycle detection returning one actual cycle; available-courses query; greedy credit-limited scheduler. 4. *Evaluation* — validate ordering/acyclicity against NetworkX; visualize graph, schedule, and any cycle; complexity write-up; runtime plot vs. course count.

Outcomes. CourseGraph + topological_sort, find_cycle, available_courses, create_schedule; a printed semester plan and a custom-vs-NetworkX check.

Main references. Goodrich et al. — **Ch. 14** (DFS, topological ordering), **Ch. 6 Queues**, **Ch. 10 Hash Tables**. · A. B. Kahn, “Topological sorting of large networks,” *CACM* 5(11), 1962. · CLRS, 4th ed.

A3. Knowledge Graph Explorer

Course anchors: labeled directed graphs, adjacency lists, hash maps, sets, BFS, DFS. **Stretch concept (new): BFS shortest-path reconstruction** (predecessor map) and **directed-cycle detection**, with rule-based Q&A.

Problem. Load facts as (subject, relation, object) triples into a labeled directed graph and explore it: look up relationships, find the shortest relationship path between two entities, detect cycles, and answer one-hop questions that always cite their supporting triples.

Dataset. A small **English ConceptNet subset** (~2,000–20,000 triples on one topic) or an **instructor-provided mini-KG CSV**.

Implementation stages. 1. *Foundation* — load/clean triples (normalize labels, drop duplicates); build a graph with outgoing **and** incoming indexes; report stats; test on a known small graph. 2. *Core algorithm* — BFS with depth tracking; exact + prefix entity search. 3. *Stretch + features* — BFS shortest path via predecessor map; DFS cycle detection returning one cycle; one-hop rule-based Q&A. 4. *Evaluation* — validate BFS/paths against NetworkX; visualize a neighborhood, a path, and a cycle; complexity write-up; runtime plot vs. graph size.

SJSU 2026 Fall CMPE 180A Week 5 Term Project Intro

Prof. Sanja Damjanovic

sanja.damjanovic@sjsu.edu

Outcomes. KnowledgeGraph + bfs, shortest_path, find_cycle, entity search, and a one-hop QA function; output that always shows supporting facts.

Main references. Goodrich et al. — **Ch. 14** (BFS, DFS), **Ch. 10 Hash Tables**, **Ch. 13** (trie for autocomplete). · CLRS, 4th ed. — BFS/DFS and unweighted shortest paths. · Speer, Chin & Havasi, “ConceptNet 5.5,” *AAAI 2017*.

A4. Social Network Analysis

Course anchors: graphs, adjacency lists, BFS, DFS, queues, hashmaps **Stretch concept (new): PageRank via power iteration**, alongside **connected-component analysis**.

Problem. Load a real social graph, report its structure, traverse it, find shortest paths, identify connected components, and rank nodes by importance with PageRank — then test whether PageRank and degree pick the same influential nodes.

Dataset. SNAP Facebook Social Circles (4,039 nodes, undirected — small enough to test and visualize). Advanced teams may sample ~1–5k nodes from the SNAP Twitter ego-network.

Implementation stages. 1. *Foundation* — load the edge list into an adjacency-list graph; handle duplicate edges/isolated nodes; report degree distribution and top-degree nodes; test on a small graph. 2. *Core algorithm* — BFS and DFS with visited sets; unweighted shortest paths over ≥ 10 node pairs. 3. *Stretch + features* — connected components; PageRank (power iteration, $d = 0.85$, convergence check, dangling-node handling); verify scores sum ≈ 1 . 4. *Evaluation* — PageRank-vs-degree comparison; validate against NetworkX; two visualizations (structure + a PageRank-sized view); runtime + convergence plots.

Outcomes. Custom Graph, BFS/DFS, shortest path, components, and pagerank; a graph summary and a PageRank-vs-degree table.

Main references. Goodrich et al. — **Ch. 14** (BFS/DFS, shortest paths, SCC), **Ch. 10 Hash Tables**, **Ch. 6 Queues**. · Page, Brin, Motwani & Winograd, “The PageRank Citation Ranking,” Stanford InfoLab, 1999. · Leskovec, Rajaraman & Ullman, *Mining of Massive Datasets*, 3rd ed., Ch. 5.

SJSU 2026 Fall CMPE 180A Week 5 Term Project Intro

Prof. Sanja Damjanovic

sanja.damjanovic@sjsu.edu

Group B — Hashing, Tries & Text Processing

B1. Autocomplete & Fuzzy Spell-Checker

Course anchors: hash maps, trees, recursion, strings. **Stretch concept (new):** the **trie** and **edit distance (Levenshtein)** via **dynamic programming**, combined for fuzzy matching.

Problem. Build an autocomplete engine that returns the top- k completions of a prefix ranked by word frequency, and that corrects misspellings by suggesting the closest real words.

Dataset. Peter Norvig's **big.txt** (~6 MB English text) plus the **SymSpell 82,765-word frequency list** — both single-file, direct download.

Implementation stages. 1. *Foundation* — parse the corpus into a frequency hashmap; build a trie supporting insert and prefix traversal; test insert/lookup. 2. *Core algorithm* — top- k completions using a heap over subtree frequencies. 3. *Stretch + features* — Levenshtein distance (DP table, checked against a recursive version); fuzzy suggestions via a bounded-edit trie walk or BK-tree. 4. *Evaluation* — trie vs. linear-scan latency across vocabulary sizes; complexity write-up; report.

Outcomes. An Autocomplete class exposing `complete(prefix, k)` and `correct(word)`; runtime plots and tests.

Main references. Goodrich et al. — **Ch. 13 Text Processing** (tries, edit-distance DP), **Ch. 10 Hash Tables**, **Ch. 4 Recursion**. · P. Norvig, "How to Write a Spelling Corrector" (norvig.com/spell-correct.html). · Burkhard & Keller, "Some approaches to best-match file searching," *CACM* 16(4), 1973 (BK-tree).

B2. Document Search Engine

Course anchors: hash maps, tries, heaps, sets, strings. **Stretch concept (new):** an **inverted index** with **trie prefix search** and **heap-based top- k ranking** (TF-IDF).

Problem. Build a search engine over a folder of plain-text or digital documents: tokenize, build an inverted index, and return the top-ranked pages for a query, with prefix autocomplete and snippets.

Dataset. ~20–50 **Project Gutenberg** .txt books (or the classic **20 Newsgroups** text collection) — open, direct download, no OCR needed.

Implementation stages. 1. *Foundation* — load documents, assign (`doc_id`, `page`) ids, implement your own tokenizer; test tokenization. 2. *Core algorithm* — build the inverted

SJSU 2026 Fall CMPE 180A Week 5 Term Project Intro

Prof. Sanja Damjanovic

sanja.damjanovic@sjsu.edu

`index dict[str, dict[(doc, page), count]]`; exact + multiword search. 3. *Stretch + features* — a trie for prefix search/autocomplete; TF-IDF ranking with heap-based top- k ; result snippets. 4. *Evaluation* — heap vs. full-sort timing; index-build/query time vs. collection size; Precision@ k on a few labeled queries; report.

Outcomes. Custom InvertedIndex and Trie + a top- k ranker; index summary, ranked results with snippets, prefix demo, and tests.

Main references. Goodrich et al. — **Ch. 10 Hash Tables, Ch. 13 (tries), Ch. 9 Priority Queues.** · Manning, Raghavan & Schütze, *Introduction to Information Retrieval* (Cambridge, 2008), Ch. 1 & 6 (free online). · Bird, Klein & Loper, *Natural Language Processing with Python* (O'Reilly, 2009).

Group C — Heaps, Ranking & Recommendation

C1. Movie Recommender with k-Nearest-Neighbors

Course anchors: hash maps, sorting/searching, heaps, recursion. **Stretch concept (new): item-based collaborative filtering with cosine similarity**, plus a **k-d tree** (or LSH) for sublinear neighbor search.

Problem. Predict how a user would rate an unseen movie and produce a top- k recommendation list, then make neighbor search fast enough to beat a brute-force all-pairs comparison.

Dataset. **MovieLens ml1-latest-small** (~100k ratings, 9k movies) — direct download; scale to MovieLens 32M for the experiment.

Implementation stages. 1. *Foundation* — load ratings into a sparse user×item structure (hashmaps of hashmaps); test loading and lookups. 2. *Core algorithm* — cosine/Pearson item similarity; top- N neighbor lists via a heap; similarity-weighted predictions. 3. *Stretch + features* — replace brute-force neighbor search with a k-d tree (or LSH) over a low-dimensional embedding. 4. *Evaluation* — RMSE and precision@ k vs. a popularity baseline; brute-force vs. k-d tree timing; report.

Outcomes. A Recommender with `predict(user, item)` and `recommend(user, k)`; an evaluation report and tests.

Main references. Goodrich et al. — **Ch. 10 Hash Tables, Ch. 9 Priority Queues, Ch. 12 Sorting and Selection, Ch. 11 Search Trees.** · Sarwar, Karypis, Konstan & Riedl, “Item-

SJSU 2026 Fall CMPE 180A Week 5 Term Project Intro

Prof. Sanja Damjanovic

sanja.damjanovic@sjsu.edu

Based Collaborative Filtering,” *WWW 2001*. · Bentley, “Multidimensional Binary Search Trees,” *CACM* 18(9), 1975. · T. Segaran, *Programming Collective Intelligence* (O’Reilly, 2007), Ch. 2.

Group D — Linked Structures & Caching

D1. Build Your Own Cache (LRU / LFU)

Course anchors: linked lists, hashmaps, queues **Stretch concept (new):** the **$O(1)$ LRU cache** (hashmap + doubly linked list) and the harder **$O(1)$ LFU cache**.

Problem. Implement LRU and LFU eviction policies that both run in $O(1)$ per access, then measure their hit rates on a real request trace and explain when each policy wins.

Dataset. **Wikimedia pageview counts** (one hourly pageviews file — direct download) as a request stream, plus a **Zipfian synthetic generator** you write for controlled experiments.

Implementation stages. 1. *Foundation* — parse a pageview slice into an ordered request sequence; implement a doubly linked list from scratch; test list operations. 2. *Core algorithm* — LRU cache (hashmap + linked list) with $O(1)$ get/put; test the $O(1)$ behavior. 3. *Stretch + features* — LFU cache in $O(1)$ (frequency buckets as linked lists); FIFO baseline. 4. *Evaluation* — hit-rate-vs-capacity curves for LRU/LFU/FIFO on the real trace and on Zipfian data; complexity write-up; report.

Outcomes. LRUCache and LFUCache with passing tests and documented $O(1)$ claims; hit-rate curves and interpretation.

Main references. Goodrich et al. — **Ch. 7 Linked Lists, Ch. 10 Hash Tables, Ch. 6 Queues, Ch. 15 Memory Management** (caching §15.1). · Shah, Mitra & Matani, “An $O(1)$ algorithm for the LFU cache eviction scheme,” 2010. · Gorelick & Ozsvald, *High Performance Python* (O’Reilly, 2020).

SJSU 2026 Fall CMPE 180A Week 5 Term Project Intro

Prof. Sanja Damjanovic

sanja.damjanovic@sjsu.edu

Group E — Streaming & Windowed Analytics

E1. Real-Time Taxi-Stream Analytics

Course anchors: sorting/searching, heaps, trees, hash maps. **Stretch concept (new):** **running median with two heaps** and a **Fenwick tree (BIT)** for rank/percentile queries over a sliding window.

Problem. Process a large stream of trip records and answer running-statistics queries — the median fare in the last N trips, top- k busiest pickup zones, and the p -th percentile trip duration — without re-sorting on every query.

Dataset. NYC TLC Trip Record Data (one month of Yellow/Green taxi Parquet — direct download; tens of millions of rows with fares, distances, timestamps, zones).

Implementation stages. 1. *Foundation* — load one month with pandas/pyarrow, clean and sort by pickup time to form the stream; test the loader on a sample. 2. *Core algorithm* — two-heap running median with $O(\log n)$ updates; validate against `numpy.median` on windows. 3. *Stretch + features* — Fenwick tree (or order-statistic BST) for percentile/rank queries; heap + hashmap top- k zones. 4. *Evaluation* — streaming structures vs. the “re-sort every query” baseline; throughput vs. window size; report.

Outcomes. A StreamStats module answering `median()`, `percentile(p)`, `top_zones(k)` incrementally; correctness checks against pandas/numpy and a throughput comparison.

Main references. Goodrich et al. — **Ch. 9 Priority Queues**, **Ch. 12 Sorting and Selection** (selection §12.4), **Ch. 11 Search Trees**. · Fenwick, “A New Data Structure for Cumulative Frequency Tables,” *SP&E* 24(3), 1994. · McKinney, *Python for Data Analysis*, 3rd ed. (O’Reilly, 2022).

E2. Chat Analytics System

Course anchors: hash maps, heaps, queues, sets, strings. **Stretch concept (new):** **heap-based streaming top- k** and a **sliding-window queue** (Count-Min Sketch as an advanced option).

Problem. Analyze an exported chat log to surface active users, trending words, and how activity moves over time — first in batch, then optionally as a real-time stream where structures update per message.

Dataset. Cornell Movie-Dialogs Corpus (~220k exchanges, via ConvoKit — direct download) or a provided ~10k–50k-message CSV (timestamp, user, message).

SJSU 2026 Fall CMPE 180A Week 5 Term Project Intro

Prof. Sanja Damjanovic

sanja.damjanovic@sjsu.edu

Implementation stages. 1. *Foundation* — ChatAnalyzer loads timestamp, user, message; count messages per user with a hashmap; return top- N users; test counts. 2. *Core algorithm* — text preprocessing (stop words via a set) and top_k_words(k) with a heap (no full-vocabulary sort). 3. *Stretch + features* — rolling message activity over a fixed window with a deque; corpus stats (unique users, mean/median length, vocabulary size, busiest hour). 4. *Evaluation* — runtime vs. number of messages; heap top- k vs. full-sort timing; (optional) real-time stream simulation; report.

Outcomes. A ChatAnalyzer with tests; a printed analytics report and a timing plot.

Main references. Goodrich et al. — **Ch. 10 Hash Tables, Ch. 9 Priority Queues, Ch. 6 Queues, Ch. 12 Sorting and Selection.** · Cormode & Muthukrishnan, “The Count-Min Sketch and its Applications,” *J. Algorithms* 55(1), 2005. · McKinney, *Python for Data Analysis*, 3rd ed. (O’Reilly, 2022).

-
- T. Cormen, C. Leiserson, R. Rivest & C. Stein, *Introduction to Algorithms*, 4th ed., MIT Press, 2022. (CLRS)
 - R. Sedgewick & K. Wayne, *Algorithms*, 4th ed., Addison-Wesley, 2011 — <https://algs4.cs.princeton.edu/>
 - J. Leskovec, A. Rajaraman & J. Ullman, *Mining of Massive Datasets*, 3rd ed., Cambridge, 2020 — <http://www.mmds.org/>
 - L. Ramalho, *Fluent Python*, 2nd ed., O’Reilly, 2022; M. Gorelick & I. Ozsvald, *High Performance Python*, 2nd ed., O’Reilly, 2020.
 - NetworkX documentation (validation and visualization only): <https://networkx.org/documentation/stable/>